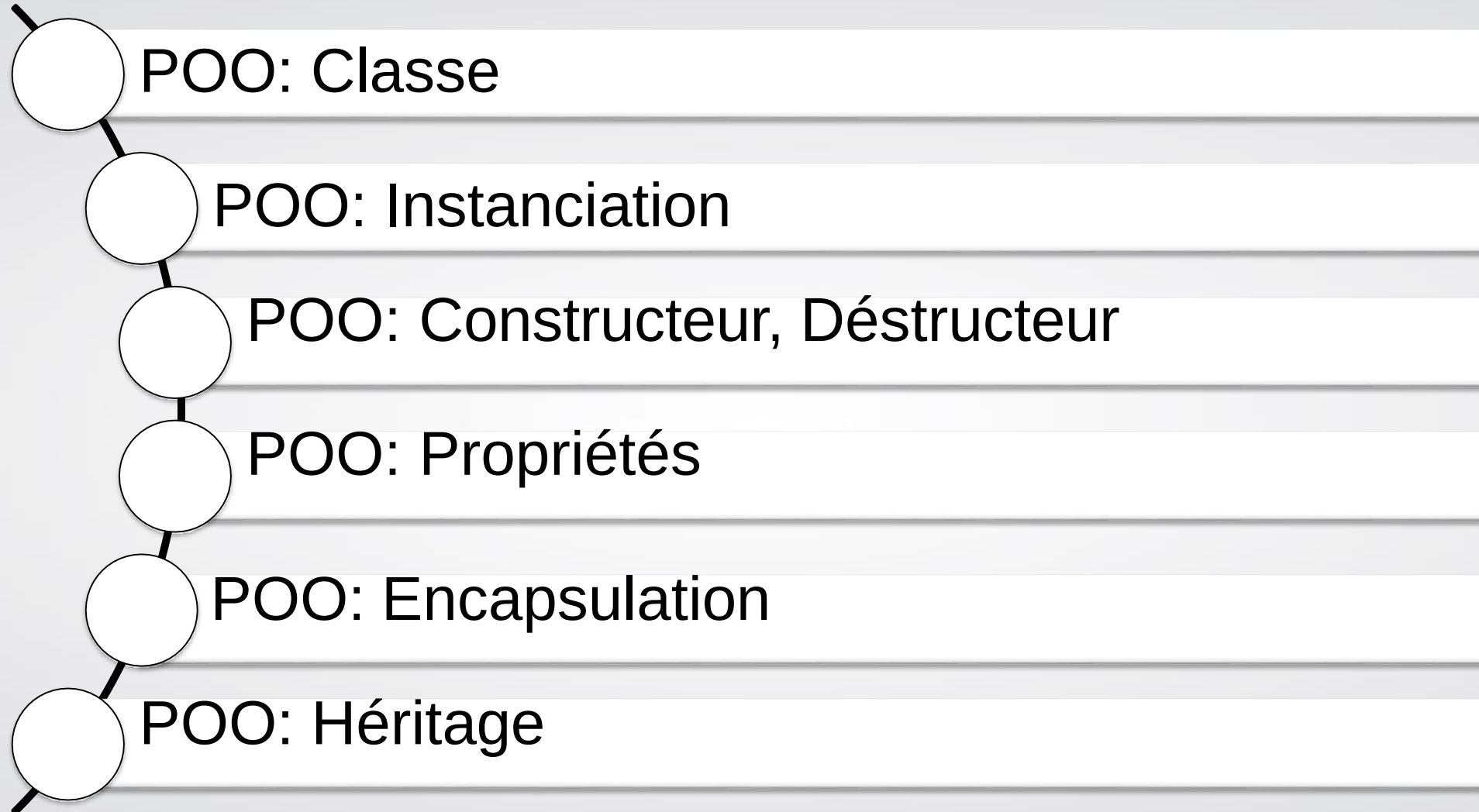


Chapitre 4: PHP

UP Web

AU: 2022/2023

Plan





Objectifs

- Les architectures du web
- Comprendre la syntaxe PHP
- Appréhender les notions de l'orientée objet
- Se connecter à une BD
- Manipuler les données d'une BD via PHP

Prérequis

- Langage HTML



► POO: Notion de Classe (1/2)

- Une **classe** est une représentation abstraite d'un **objet**.
- Une classe peut être rendue concrète au moyen d'une **instance de classe**, que l'on appelle **objet**.
- Une classe s'écrit au moyen du mot "**class**" suivi du nom de la classe et d'accolades.

```
class Personne {  
    public string $nom;  
    public string $prenom;  
  
    public function saisir(string $nom, string $prenom) { ...  
    }  
  
    public function afficher(){ ...  
    }  
}
```



POO: Notion de Classe (2/2)

Personne
+nom +prenom
+saisir(var,var) +afficher()

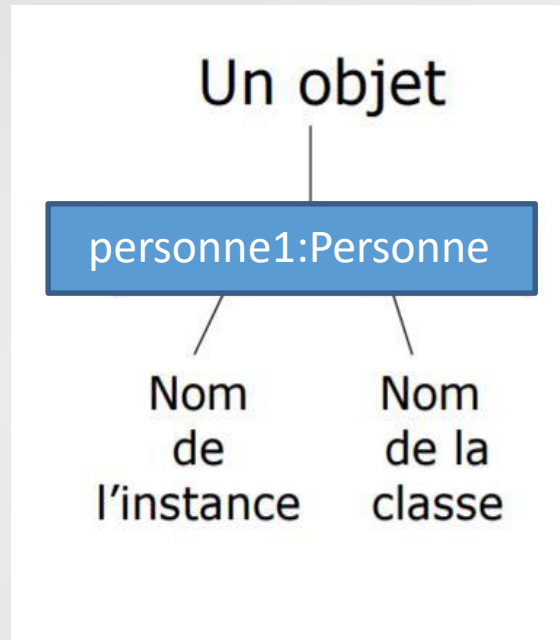
```
<?php
class Personne {
    public string $nom;
    public string $prenom;

    public function saisir(string $nom, string $prenom) {
        $this->nom = $nom;
        $this->prenom = $prenom;
    }

    public function afficher(){
        echo "Nom: $this->nom <br>";
        echo "Prenom: $this->prenom <br>";
    }
}
?>
```



► POO: Instanciation d'un objet



```
// Déclaration d'une instance
$personne1 = new Personne();

// Utilisation
$personne1->saisir('nom1', 'prenom1');

// Vérification
if ($personne1 instanceof Personne) {
    echo "L'objet \$personne1 est de type Personne. <br>";
}

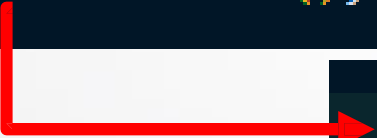
// Affichage
$personne1->afficher();
```



POO: Constructeur (1/4)

- Le constructeur est la méthode qui va être appelée à l'instanciation de l'objet.
- Il doit être implémenté dans la classe elle-même de la manière suivante :

```
// Déclaration d'une instance  
$personne1 = new Personne();
```



```
public function __construct () {  
    $this->nom = 'nom 1';  
    $this->prenom = 'prenom 1';  
}
```



► POO: Constructeur (2/4)

- Avant PHP 8, pour définir une propriété d'objet avec une valeur par défaut, il fallait le faire en deux étapes : d'abord définir la propriété, puis dans le constructeur, lui assigner une valeur par défaut
- En PHP 8, on peut combiner les deux étapes en une seule grâce à la promotion de propriété. Voici un exemple :

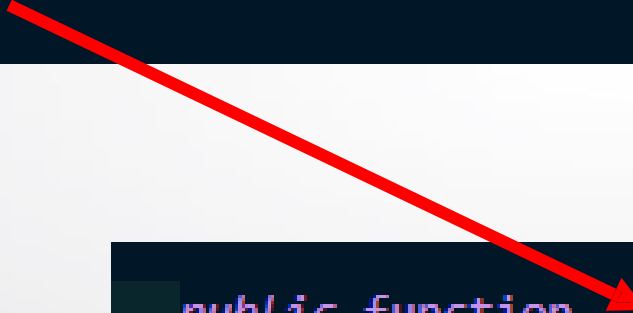
```
class Personne {  
    public function __construct(  
        public $nom = 'nom 1',  
        public $prenom = 'prenom 1') {  
    }  
}
```



► POO: Constructeur (3/4)

- Le constructeur peut aussi comporter des paramètres.
- En PHP, on ne peut avoir qu'un seul constructeur par classe.

```
// Déclaration d'une instance  
$personne1 = new Personne('nom1', 'prenom1');
```



```
public function _construct (string $nom, string $prenom) {  
    $this->nom = $nom;  
    $this->prenom = $prenom;  
}
```



POO: Constructeur (4/4)

- Dans PHP 7, un constructeur paramétré prend un argument non typé
- PHP 8 utilise la fonctionnalité de typage des arguments de PHP 8 pour typer l'argument du constructeur et la propriété de l'objet.
- Le typage des arguments et des propriétés est une fonctionnalité utile pour s'assurer que les valeurs passées à une fonction ou stockées dans une propriété sont du bon type et pour éviter les erreurs lors de l'exécution du code.



► POO: Déstructeur

- Un destructeur d'une classe donnée est une méthode exécutée automatiquement à chaque fois qu'une instance de la classe donnée disparaît.
- C'est une méthode sans type de retour
- C'est une méthode sans paramètre, elle ne peut donc pas être surchargée.
- C'est une méthode en accès public.

```
public function __destruct(){  
    echo "Je suis le destructeur";  
}
```

```
// Affichage  
$personne1->afficher();  
  
unset($personne1);
```



► POO: Remarque (1/2)

Constructeur

- ✓ **PHP 3 et 4** : une fonction portant le même nom que la classe
- ✓ **PHP5, PHP7 ou supérieur** : une fonction membre spécifique. PHP5 ne permet pas la **surcharge(overloading)** de fonction (ou de méthodes) et donc on ne peut attribuer le même nom à plusieurs fonctions. Par contre la redéfinition est possible.

Destructeur

- ✓ **PHP 3 et 4** : il n'y a pas de destructeur
- ✓ **PHP 5, PHP7 et supérieur**: introduit la notion de destructeur. On utilise la fonction `unset($MonPoint)`.



► POO: Remarque (2/2)

unset()

unset(\$nom) détruit une variable

```
//détruire une variable
```

```
unset($x);
```

```
//détruire une case d'un tableau
```

```
unset($t[0]);
```

```
//détruire plusieurs variables
```

```
unset($a, $b, $c);
```



► POO: Propriétés d'un objet (1/2)

- Une propriété est une variable associée à un objet.
- On peut demander « le nom de cette personne » et non le nom en général.

```
$personne1 = new Personne();  
var_dump($personne1->nom);
```

→ Pas de caractère \$ devant le nom de la propriété.



► POO: Propriétés d'un objet (2/2)

- Une propriété est une variable associée à un objet.
- On peut demander « le nom de cette personne » et non le nom en général.

```
$personne1 = new Personne();  
var_dump($personne1->nom);
```



La propriété nom est liée intrinsèquement à l'objet.
Cette liaison est notée par la flèche.



POO: Droits d'accès

- Les méthodes et les variables “ **(+)public**” sont visibles et manipulables par tous les objets, même s'ils sont relatifs à d'autres classes.
- Les méthodes et les variables “ **(#)protected**” concernent les objets de la même classe ainsi que ses dérivés, mais pas ceux des classes étrangères.
- Les classes ont des variables et des méthodes internes et qui ne concernent pas l'extérieur. Ces propriétés sont déclarées en tant que “ **(-)private**”.



POO: Encapsulation (1/2)

- L'encapsulation est la pratique consistant à regrouper des attributs au sein d'une même classe.
 - Pour améliorer la lisibilité des programmes, les attributs encapsulés sont souvent **privés** (inaccessibles aux autres classes)
- Les données et méthodes accessibles sont dites **publiques**
- L'encapsulation peut être renforcée en utilisant des types de propriété pour typer les propriétés d'une classe et en utilisant des types de retour pour typer les méthodes d'une classe.

POO: Encapsulation (2/2)



```
class Personne {  
    private string $nom;  
    private string $prenom;  
    private int $age;  
  
    public function getNom(): string  
    {  
        return $this->nom;  
    }  
  
    public function setNom(string $nom)  
    {  
        $this->nom = $nom;  
        return $this;  
    }  
  
    public function getPrenom(): string  
    {  
        return $this->prenom;  
    }  
  
    public function setPrenom(string $prenom)  
    {  
        $this->prenom = $prenom;  
        return $this;  
    }  
  
    public function getAge(): int  
    {  
        return $this->age;  
    }  
  
    public function setAge(int $age)  
    {  
        $this->age = $age;  
        return $this;  
    }  
}
```

Accesseurs (Getters)

permettent de
renvoyer la valeur
d'une propriété.

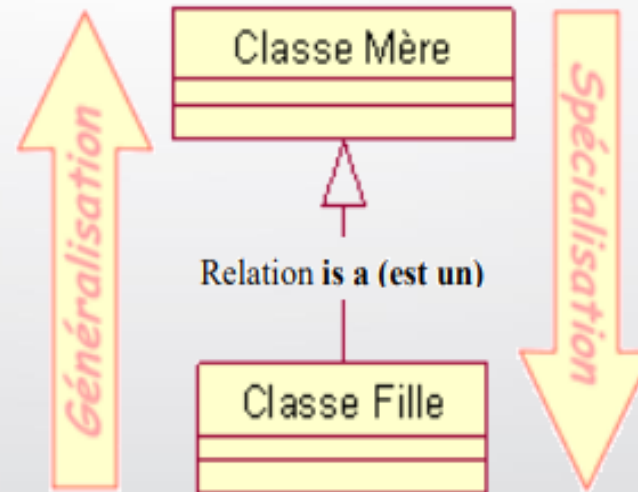
Mutateurs (Setters)

permettent de
modifier la valeur
d'une propriété.



POO: Héritage

- L'héritage est un concept de la programmation orientée objet qui permet à une classe d'hériter des propriétés et des méthodes d'une classe parente
- Une classe générale définit alors un ensemble d'attributs et/ou méthodes qui sont partagés par d'autres classes, dont on dira qu'elles héritent de cette classe générale.





► POO: Heritage de constructeurs

- En PHP 8, on peut définir des constructeurs dans les classes filles qui appellent le constructeur de la classe parente avec des arguments différents,

comme ceci :

```
class MaClasseParente {  
    public function __construct(string $arg1, int $arg2) {  
        // ...  
    }  
}  
  
class MaClasseFille extends MaClasseParente {  
    public function __construct(int $arg1) {  
        parent::__construct('valeur par défaut', $arg1);  
    }  
}
```



POO: Redéfinition

- PHP permet la redéfinition des méthodes, c'est-à-dire la possibilité de re-déclarer les mêmes attributs et opérations d'une super classe au sein d'une sous classe. Nous pouvons aussi modifier la signature de la fonction, et son traitement en la remplaçant par notre propre implémentation.

```
class Personne {  
    private $nom;  
    private $prenom;  
    private $age;  
  
    public function afficher() {  
        echo "Nom : " . $this->nom . "<br>";  
        echo "Prénom : " . $this->prenom . "<br>";  
        echo "Âge : " . $this->age . "<br>";  
    }  
}
```

```
class Etudiant extends Personne {  
    private $annee;  
  
    public function afficher() {  
        parent::afficher();  
        echo "Année d'étude : " . $this->annee . "<br>";  
    }  
}
```



L'architecture MVC

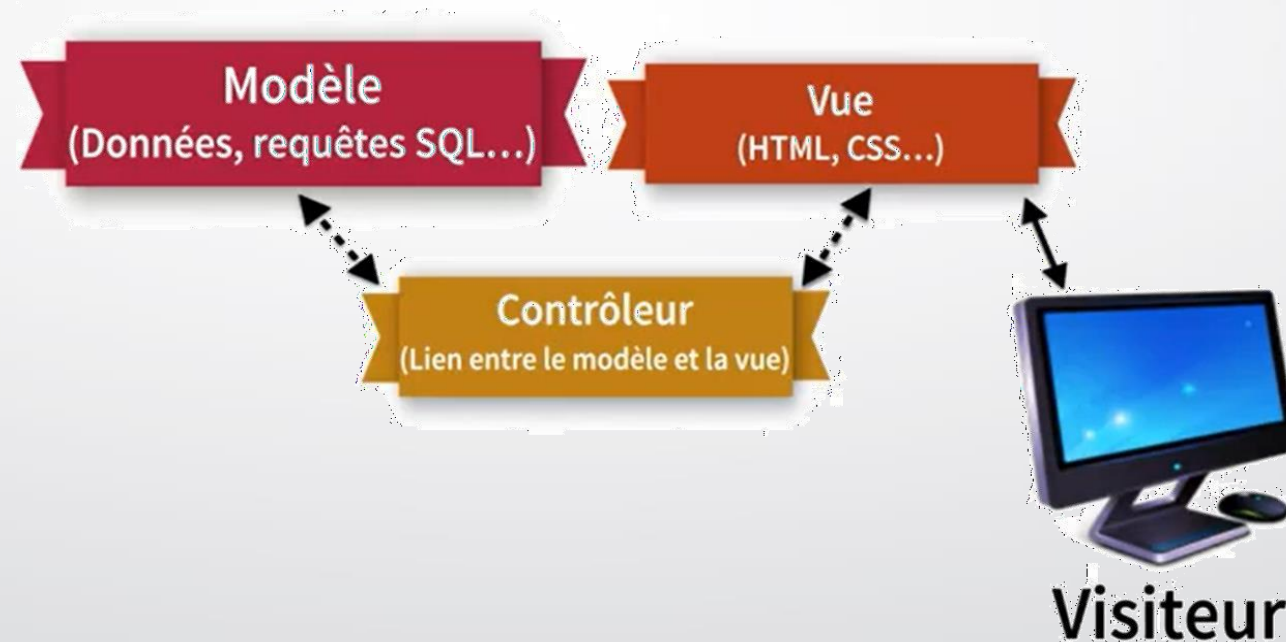


Le modèle MVC (Modèle-Vue-Contrôleur) est un motif de conception couramment utilisé en développement logiciel pour séparer l'interface utilisateur, de gestion de données et de traitement de la logique applicative dans une application

- *Aide à organiser et structurer le code d'une application web.

- *Facilite la maintenance et l'évolution de l'application.

=>En PHP, cela peut se faire en créant des classes distinctes pour chaque partie du modèle MVC

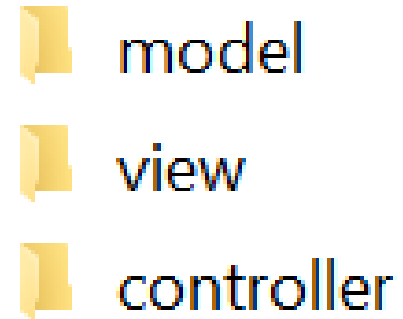




L'architecture MVC

- 1. Le modèle (Model)** : représente les données de l'application et contient la logique qui permet de les manipuler.
- 2. La vue (View)** : représente l'interface utilisateur de l'application. Elle affiche les données du modèle de manière compréhensible pour l'utilisateur
- 3. Le contrôleur (Controller)** : gère les interactions entre le modèle et la vue. Il récupère les données du modèle et les envoie à la vue pour les afficher.

.



- model
- view
- controller



▶ Les fonctions 'include' et 'require'

Pour inclure un fichier PHP dans un autre fichier, vous pouvez utiliser la fonction include ou require en PHP

Include() : Cette fonction permet d'inclure un fichier PHP dans un autre fichier sans générer une erreur fatale si le fichier inclus n'est pas trouvé. La syntaxe est la suivante :

```
include 'chemin/vers/fichier.php';
```

ou

```
include_once 'chemin/vers/fichier.php';
```

L'utilisation de **include_once()** permet de s'assurer que le fichier n'est inclus qu'une seule fois, même si vous avez plusieurs appels à include dans votre code.



▶ Les fonctions 'include' et 'require'

- La différence entre `include` et `include_once`:
 - `include_once` inclut et évalue le fichier spécifié durant l'exécution du script. Le comportement est similaire à `include`, mais la différence est que si le code a déjà été inclus, il ne le sera pas une seconde fois, et `include_once` retourne TRUE.
 - La structure `include_once` est utilisée:
 - de préférence lorsque le fichier va être inclus ou évalué plusieurs fois dans un script,
 - ou bien lorsque vous voulez être sûr qu'il ne sera inclus qu'une seule fois, pour éviter des redéfinitions de fonctions ou de classes.



▶ Les fonctions 'include' et 'require'

require() : Cette fonction permet d'inclure un fichier PHP dans un autre fichier, mais génère une erreur fatale si le fichier inclus n'est pas trouvé. La syntaxe est la suivante :

```
require 'chemin/vers/fichier.php';
```

ou

```
require_once 'chemin/vers/fichier.php';
```



► Les fonctions 'include' et 'require'

- La différence entre `include` et `require`:
 - `require` est identique à `include` mise à part le fait que lorsqu'une erreur survient, il produit également une erreur fatale de niveau `E_COMPILE_ERROR`.
 - Il stoppera le script alors que `include` n'émettra qu'une alerte de niveau `E_WARNING`, ce qui permet au script de continuer.



Les fonctions 'include' et 'require'

- La différence entre `require` et `require_once`?
 - L'instruction `require_once` est identique à `require` mis à part que PHP vérifie si le fichier a déjà été inclus, et si c'est le cas, ne l'inclut pas une deuxième fois.



 **Merci de votre attention**